18-661 Introduction to Machine Learning

Nearest Neighbors

Spring 2026

ECE – Carnegie Mellon University

Announcements

- Homework 2 is due on Friday February 20. Homework 3 will be released on Friday as well.

Announcements

- Homework 2 is due on Friday February 20. Homework 3 will be released on Friday as well.
- Midterm exam is on Wednesday, February 25 in lecture.
  - The end of this lecture has some slides on mini-exam 1 and review for the midterm.
  - Friday's recitation will go over practice problems and exam review.

Outline

1. Recap: SVMs
2. Parametric and Nonparametric Models
3. Nearest Neighbor Classification
4. K Nearest Neighbors Classification
5. Practical Aspects of NN
6. Midterm Review

Recap: SVMs

Summary: Three SVM Formulations

Hard-margin (for separable data)

$$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|_2^2 \ \text{s.t.} \ y_n[\boldsymbol{w}^\top \boldsymbol{x}_n + b] \geq 1, \ \forall \ n$$

Soft-margin (add slack variables)

to handle non-separable data

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\boldsymbol{w}\|_2^2 + C\sum_n \xi_n \ \ \text{s.t.} \ y_n[\boldsymbol{w}^\top \boldsymbol{x}_n + b] \geq 1 - \xi_n, \ \xi_n \geq 0, \ \forall \ n$$

Hinge loss (define a loss function for each data point)

$$\min_{\boldsymbol{w},b} \sum_n \underbrace{\max(0, 1 - y_n[\boldsymbol{w}^\top \boldsymbol{x}_n + b])}_{\text{hinge loss}} + \frac{\lambda}{2}\|\boldsymbol{w}\|_2^2$$

training data

regularizer

Summary of Dual Formulation

Primal Max-Margin Formulation

soft margin

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \quad \frac{1}{2}\|\boldsymbol{w}\|_2^2 + C\sum_n \xi_n$$

$$\text{s.t.} \quad y_n[\boldsymbol{w}^\top \boldsymbol{x}_n + b] \geq 1 - \xi_n, \quad \forall \ n$$

$$\xi_n \geq 0, \quad \forall \ n$$

duality

equivalent

Dual Formulation

$$\max_{\boldsymbol{\alpha}} \quad \sum_n \alpha_n - \frac{1}{2}\sum_{m,n} y_m y_n \alpha_m \alpha_n \boldsymbol{x}_m^\top \boldsymbol{x}_n$$

$$\text{s.t.} \quad 0 \leq \alpha_n \leq C, \quad \forall \ n$$

$$\sum_n \alpha_n y_n = 0$$

dual variables

convex quadratic program

- In dual formulation, the # of variables is independent of dimension.
- Only the support vectors have nonzero dual variables.
- Can easily recover the primal solution $\boldsymbol{w}, b$ from dual solution.

Primal and Dual SVM Formulations: Kernel Versions

nonlinear basis function $\phi(x)$

Primal

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \quad \frac{1}{2}\|\boldsymbol{w}\|_2^2 + C\sum_n \xi_n$$

$$\text{s.t.} \quad y_n[\boldsymbol{w}^\top \phi(\boldsymbol{x}_n) + b] \geq 1 - \xi_n, \quad \forall \ n$$

$$\xi_n \geq 0, \quad \forall \ n$$

Dual

$$\max_{\boldsymbol{\alpha}} \sum_n \alpha_n - \frac{1}{2}\sum_{m,n} y_m y_n \alpha_m \alpha_n \boxed{\phi(\boldsymbol{x}_m)^\top \phi(\boldsymbol{x}_n)}$$

only need inner product

$$\text{s.t.} \quad 0 \leq \alpha_n \leq C, \quad \forall \ n$$

$$\sum_n \alpha_n y_n = 0$$

IMPORTANT POINT: In the dual problem, we only need $\phi(\boldsymbol{x}_m)^\top \phi(\boldsymbol{x}_n)$.

The Kernel Trick

In dual SVM, we can use any of the kernel functions discussed in the previous lecture.

radial basis kernel

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(\boldsymbol{x}_m, \boldsymbol{x}_n) \\ \text{s.t.} \quad & 0 \leq \alpha_n \leq C, \quad \forall\ n \\ & \sum_n \alpha_n y_n = 0 \end{aligned}$$

The Kernel Trick

In dual SVM, we can use any of the kernel functions discussed in the previous lecture.

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(\boldsymbol{x}_m, \boldsymbol{x}_n) \\ \text{s.t.} \quad & 0 \leq \alpha_n \leq C, \quad \forall\ n \\ & \sum_n \alpha_n y_n = 0 \end{aligned}$$

Each choice of kernel function will correspond to doing SVM using the transformed data $\phi(\boldsymbol{x})$, but we do not need to know what exactly is $\phi(\boldsymbol{x})$.

The Kernel Trick

In dual SVM, we can use any of the kernel functions discussed in the previous lecture.

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(\boldsymbol{x}_m, \boldsymbol{x}_n) \\ \text{s.t.} \quad & 0 \leq \alpha_n \leq C, \quad \forall\ n \\ & \sum_n \alpha_n y_n = 0 \end{aligned}$$

Each choice of kernel function will correspond to doing SVM using the transformed data $\phi(\boldsymbol{x})$, but we do not need to know what exactly is $\phi(\boldsymbol{x})$.

This allows us to use more complicated $\phi(\boldsymbol{x})$ (like the $\phi(\boldsymbol{x})$ associated with the radial basis function) to boost performance - without knowing what $\phi(\boldsymbol{x})$ is! This is known as the "kernel trick".

Test Prediction

Learning $\boldsymbol{w}$ and b:

$$\boldsymbol{w} = \sum_n \alpha_n y_n \phi(\boldsymbol{x}_n),$$

only need kernel function

$$b = y_n - \boldsymbol{w}^\top \phi(\boldsymbol{x}_n) = y_n - \sum_m \alpha_m y_m k(\boldsymbol{x}_m, \boldsymbol{x}_n)$$

But for test prediction on a new point $\mathbf{x}$, do we need the form of $\phi(\mathbf{x})$ in order to find the sign of $\boldsymbol{w}^\top \phi(\boldsymbol{x}) + b$?

Test Prediction

Learning w and b:

$$\boldsymbol{w} = \sum_n \alpha_n y_n \boldsymbol{\phi}(\boldsymbol{x}_n),$$

$$b = y_n - \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n) = y_n - \sum_m \alpha_m y_m k(\boldsymbol{x}_m, \boldsymbol{x}_n)$$

But for test prediction on a new point $\mathbf{x}$, do we need the form of $\phi(\mathbf{x})$ in order to find the sign of $\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}) + b$? Fortunately, no!

Test Prediction:

$$h(\boldsymbol{x}) = \text{SIGN}\left(\sum_n y_n \alpha_n k(\boldsymbol{x}_n, \boldsymbol{x}) + b\right)$$

dual soln

from the dual solutio

At test time it suffices to know the kernel function! So we really do not need to know ϕ.

Summary of Kernel SVM

Given a dataset $\{(\boldsymbol{x}_n, y_n)$ for $n = 1, 2, \ldots, N\}$, how do you classify it using kernel SVM ?

Select a kernel. In general, you can just use one of the popular kernel functions (polynomial kernel or radial kernel).

Summary of Kernel SVM

Given a dataset $\{(\boldsymbol{x}_n, y_n)$ for $n = 1, 2, \ldots, N\}$, how do you classify it using kernel SVM ?

Select a kernel. In general, you can just use one of the popular kernel functions (polynomial kernel or radial kernel).

Training

$$\max_{\boldsymbol{\alpha}} \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(\mathbf{x}_m, \mathbf{x}_m)$$

$$\text{s.t.} \quad 0 \leq \alpha_n \leq C, \quad \forall\ n$$

$$\sum_n \alpha_n y_n = 0$$

$\Rightarrow \{\alpha_n\}$ solution

Summary of Kernel SVM

Given a dataset $\{(\boldsymbol{x}_n, y_n)$ for $n = 1, 2, \ldots, N\}$, how do you classify it using kernel SVM ?

Select a kernel. In general, you can just use one of the popular kernel functions (polynomial kernel or radial kernel).

Training

$$\max_{\boldsymbol{\alpha}} \sum_n \alpha_n - \frac{1}{2} \sum_{m,n} y_m y_n \alpha_m \alpha_n k(\mathbf{x}_m, \mathbf{x}_m)$$

$$\text{s.t.} \quad 0 \leq \alpha_n \leq C, \quad \forall\ n$$

$$\sum_n \alpha_n y_n = 0$$

Prediction

$$h(\boldsymbol{x}) = \text{SIGN}(\sum_n y_n \alpha_n k(\boldsymbol{x}_n, \boldsymbol{x}) + b)$$

solution

training data

Advantages of SVM

Now we have shown all of the below.

1. Maximizes distance of training data from the boundary
2. Only requires a subset of the training points. support vectors
3. Is less sensitive to outliers. soft margin $\Leftrightarrow$ hinge loss $\Leftrightarrow$ dual
4. Scales better with high-dimensional data. duality
5. Generalizes well to many nonlinear models. kernel trick

Outline

1. Recap: SVMs
2. Parametric and Nonparametric Models
3. Nearest Neighbor Classification
4. K Nearest Neighbors Classification
5. Practical Aspects of NN
6. Midterm Review

Parametric and Nonparametric Models

Parametric vs. Nonparametric

- So far, we've discussed parametric machine learning models:
  - Linear regression
  - Naïve Bayes
  - Logistic regression
  - Linear SVMs

Parametric vs. Nonparametric

- So far, we've discussed parametric machine learning models:
  - Linear regression
  - Naïve Bayes
  - Logistic regression
  - Linear SVMs
- Next, we will discuss two *nonparametric* models:
  - Nearest neighbors
  - Decision trees

Parametric vs. Nonparametric

Key difference:

- Parametric models can be characterized via some *fixed* set of parameters θ. Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e., $P(x|\theta, \mathcal{D}) = P(x|\theta)$.

Parametric vs. Nonparametric

Key difference:

- Parametric models can be characterized via some *fixed* set of parameters θ. Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e., $P(x|\theta, \mathcal{D}) = P(x|\theta)$.
  - E.g. logistic regression, SVM make a prediction based on the decision boundary $\boldsymbol{w}^\top \boldsymbol{x} + b$, where $(\boldsymbol{w}, b)$ can be viewed as the parameter θ.
  - Often simpler and faster to learn, but can sometimes be a poor fit

Parametric vs. Nonparametric

Key difference:

- Parametric models can be characterized via some *fixed* set of parameters θ. Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e., $P(x|\theta, \mathcal{D}) = P(x|\theta)$.
  - E.g. logistic regression, SVM make a prediction based on the decision boundary $\boldsymbol{w}^\top \boldsymbol{x} + b$, where $(\boldsymbol{w}, b)$ can be viewed as the parameter θ.
  - Often simpler and faster to learn, but can sometimes be a poor fit
- Nonparametric models make a prediction directly based on the data $\mathcal{D}$ (without explicitly learning a fixed parameter set θ).
  - More complex and computationally expensive, but can learn more flexible patterns

Parametric vs. Nonparametric

Key difference:

- Parametric models can be characterized via some *fixed* set of parameters θ. Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e., $P(x|\theta, \mathcal{D}) = P(x|\theta)$.
  - E.g. logistic regression, SVM make a prediction based on the decision boundary $\boldsymbol{w}^\top \boldsymbol{x} + b$, where $(\boldsymbol{w}, b)$ can be viewed as the parameter θ.
  - Often simpler and faster to learn, but can sometimes be a poor fit
- Nonparametric models make a prediction directly based on the data $\mathcal{D}$ (without explicitly learning a fixed parameter set θ).
  - More complex and computationally expensive, but can learn more flexible patterns
- Both parametric and nonparametric methods can be used for either regression or classification. Both require training data with input features and labels (i.e., supervised learning).

Nearest Neighbor Classification

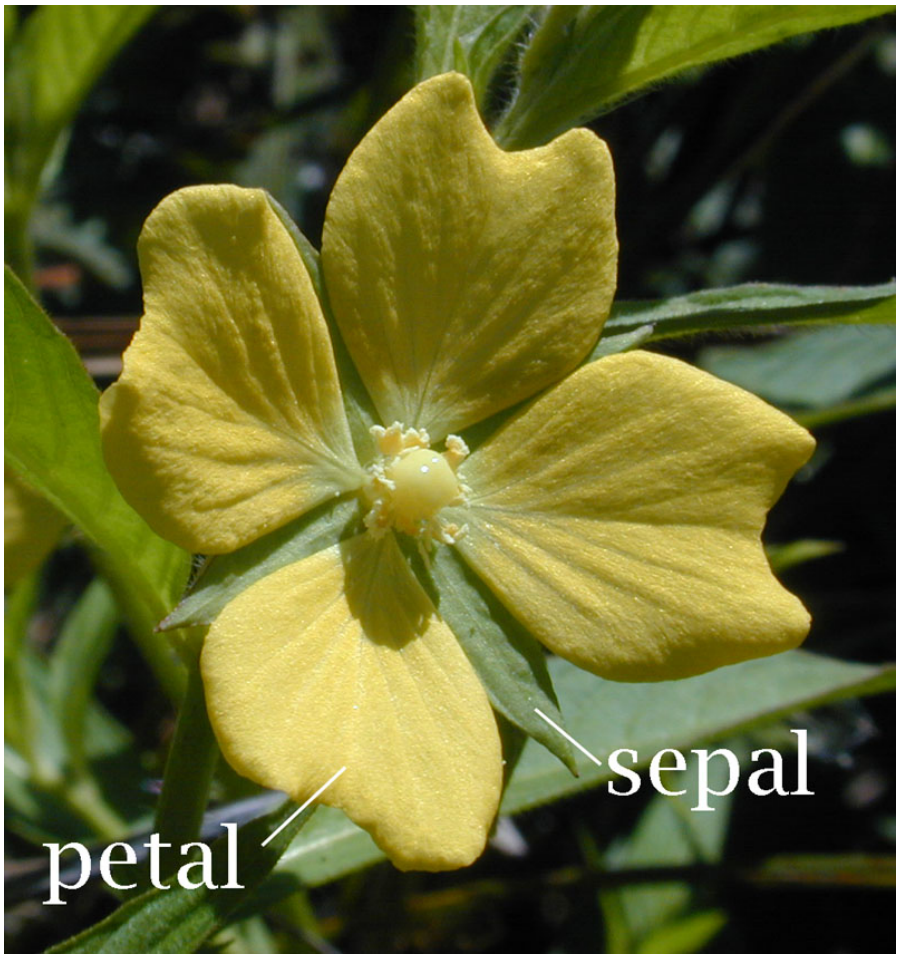
Recognizing Flowers

Fisher Iris

Types of Iris: setosa, versicolor, and virginica

Measuring Flower Properties

Features: the widths and lengths of sepal and petal



Data Often Fits into a Table

Ex: Iris data (click here for all data)

- 4 features width, length of sepal & petals
- 3 classes

Fisher's *Iris* Data

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	*I. setosa*
4.9	3.0	1.4	0.2	*I. setosa*
4.7	3.2	1.3	0.2	*I. setosa*
4.6	3.1	1.5	0.2	*I. setosa*
5.0	3.6	1.4	0.2	*I. setosa*
5.4	3.9	1.7	0.4	*I. setosa*
4.6	3.4	1.4	0.3	*I. setosa*
5.0	3.4	1.5	0.2	*I. setosa*
4.4	2.9	1.4	0.2	*I. setosa*
4.9	3.1	1.5	0.1	*I. setosa*

data point →

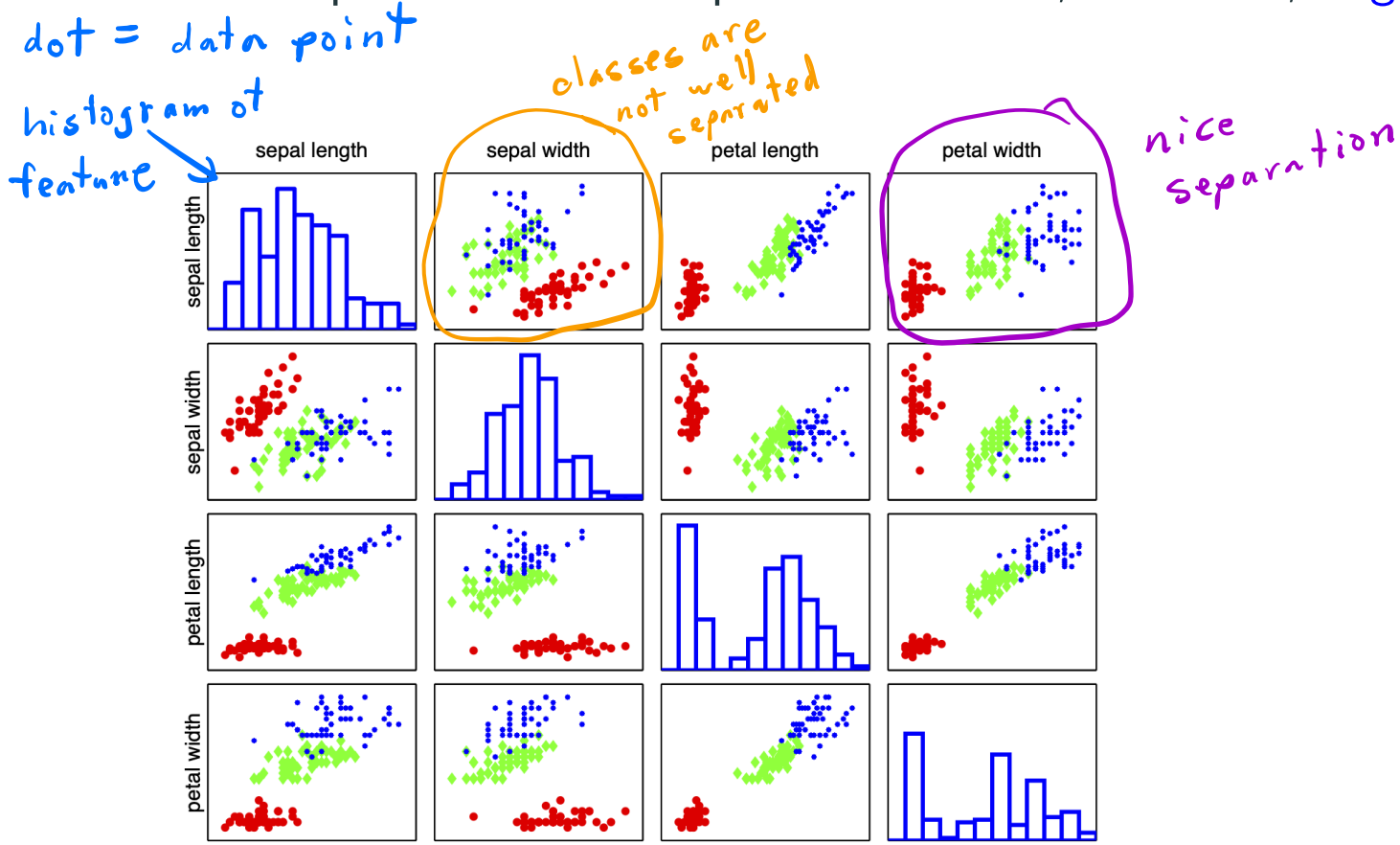
4 input features

Pairwise Scatter Plots of 131 Flower Specimens

Visualization of data helps to identify the right learning model

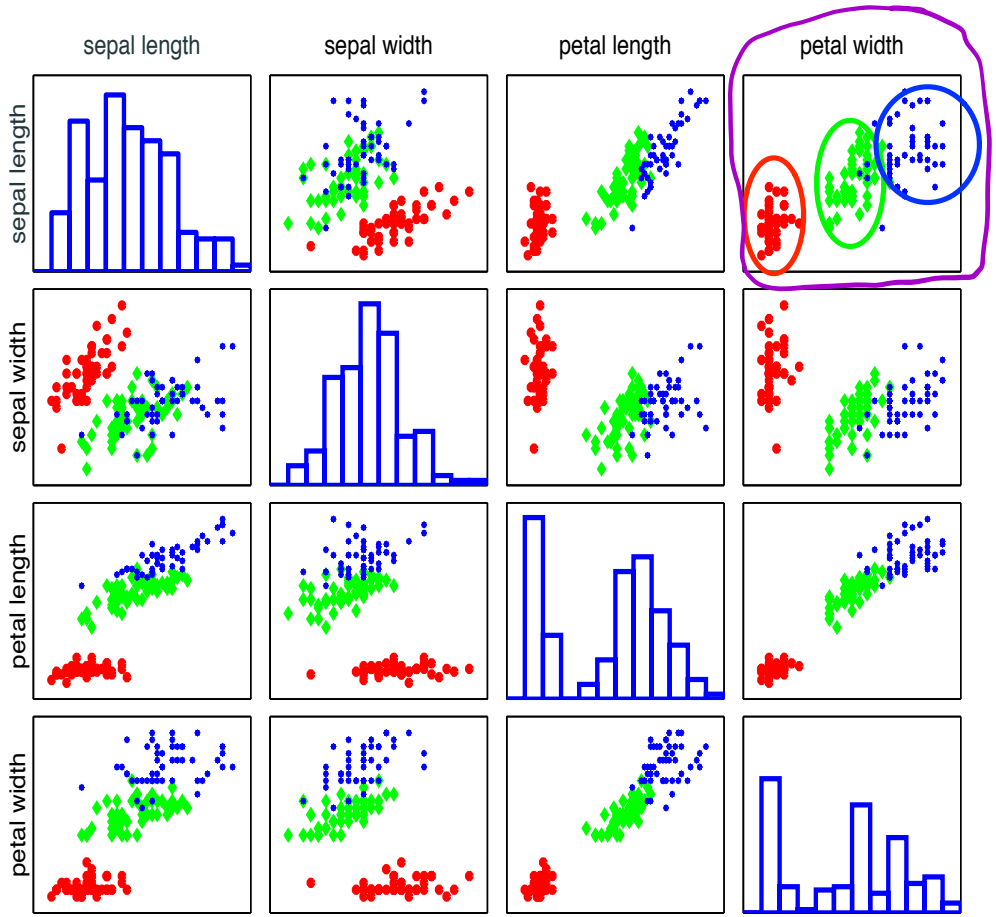
Which combination of features separates the three classes?

Figure 1: Each colored point is a flower specimen: setosa, versicolor, virginica

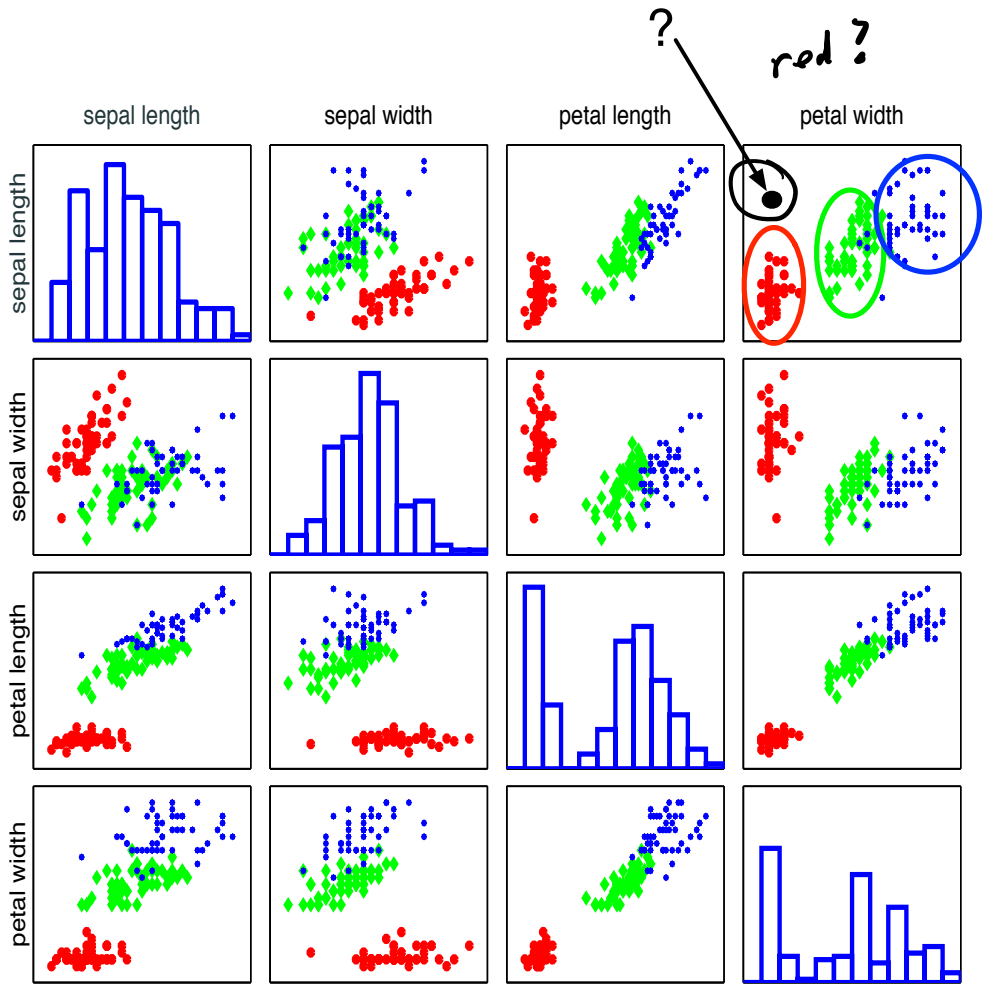


Different Types Seem Well-clustered and Separable

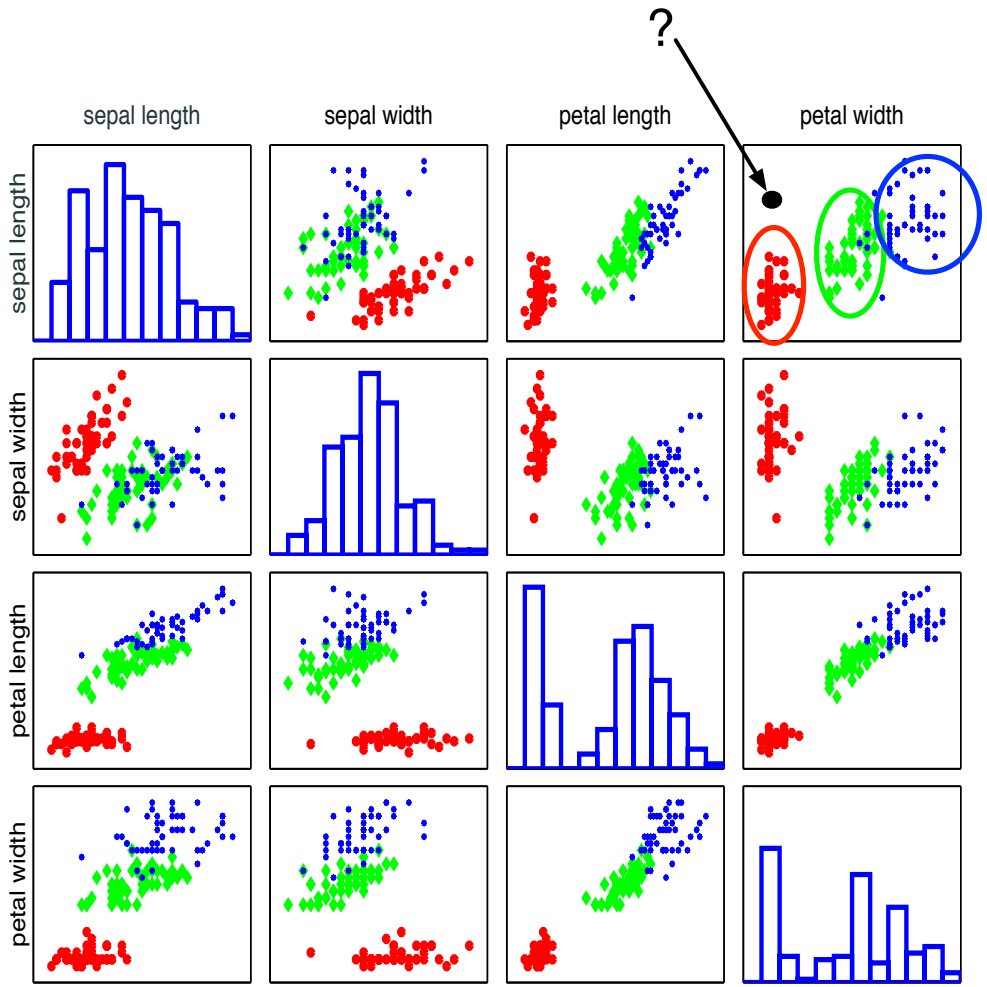
Using two features: petal width and sepal length



Labeling an Unknown Flower Type



Labeling an Unknown Flower Type



Closer to red cluster: so labeling it as setosa

Nearest Neighbor Classification (NNC)

Training data (set)

inputs / features

- N samples: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_{\mathsf{N}}, y_{\mathsf{N}})\}$
- Input $\boldsymbol{x}_n \in \mathbb{R}^{\mathsf{D}}$, output (label): $y_n \in [\mathsf{C}] = \{1, 2, \cdots, \mathsf{C}\}$
- Learning goal: given a test point $\boldsymbol{x}$, predict its label y.

Nearest Neighbor Classification (NNC)

Training data (set)

- N samples: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_\text{N}, y_\text{N})\}$
- Input $\boldsymbol{x}_n \in \mathbb{R}^\text{D}$, output (label): $y_n \in [\text{C}] = \{1, 2, \cdots, \text{C}\}$
- Learning goal: given a test point $\boldsymbol{x}$, predict its label y.

Nearest neighbor of a test data point x

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\text{nn}(\boldsymbol{x})}$$

index of x's nearest neighbor

where $\text{nn}(\boldsymbol{x}) \in [\text{N}] = \{1, 2, \cdots, \text{N}\}$ is index of the closest training sample

based on features

Nearest Neighbor Classification (NNC)

Training data (set)

- N samples: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_\text{N}, y_\text{N})\}$
- Input $\boldsymbol{x}_n \in \mathbb{R}^\text{D}$, output (label): $y_n \in [\text{C}] = \{1, 2, \cdots, \text{C}\}$
- Learning goal: given a test point $\boldsymbol{x}$, predict its label y.

Nearest neighbor of a test data point x

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\text{nn}(\boldsymbol{x})}$$

where $\text{nn}(\boldsymbol{x}) \in [\text{N}] = \{1, 2, \cdots, \text{N}\}$ is index of the closest training sample

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \underbrace{\|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2}_{\substack{\text{squared} \\ \text{Euclidean} \\ \text{distance}}} = \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} (x_d - x_{nd})^2$$

Nearest Neighbor Classification (NNC)

Training data (set)

- N samples: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_\text{N}, y_\text{N})\}$
- Input $\boldsymbol{x}_n \in \mathbb{R}^\text{D}$, output (label): $y_n \in [\text{C}] = \{1, 2, \cdots, \text{C}\}$
- Learning goal: given a test point $\boldsymbol{x}$, predict its label y.

Nearest neighbor of a test data point x

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\text{nn}(\boldsymbol{x})}$$

where $\text{nn}(\boldsymbol{x}) \in [\text{N}] = \{1, 2, \cdots, \text{N}\}$ is index of the closest training sample

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2 = \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} (x_d - x_{nd})^2$$

Classification rule

use the label of nearest neighbor

$$y = f(\boldsymbol{x}) = y_{\text{nn}(\boldsymbol{x})}$$

Nearest Neighbor Classification (NNC)

Training data (set)

- N samples: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_{\text{N}}, y_{\text{N}})\}$
- Input $\boldsymbol{x}_n \in \mathbb{R}^{\text{D}}$, output (label): $y_n \in [\text{C}] = \{1, 2, \cdots, \text{C}\}$
- Learning goal: given a test point $\boldsymbol{x}$, predict its label y.

Nearest neighbor of a test data point x

1st closest neighbor

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\text{nn}(\boldsymbol{x})}$$

where $\text{nn}(\boldsymbol{x}) \in [\text{N}] = \{1, 2, \cdots, \text{N}\}$ is index of the closest training sample

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2 = \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} (x_d - x_{nd})^2$$

Classification rule

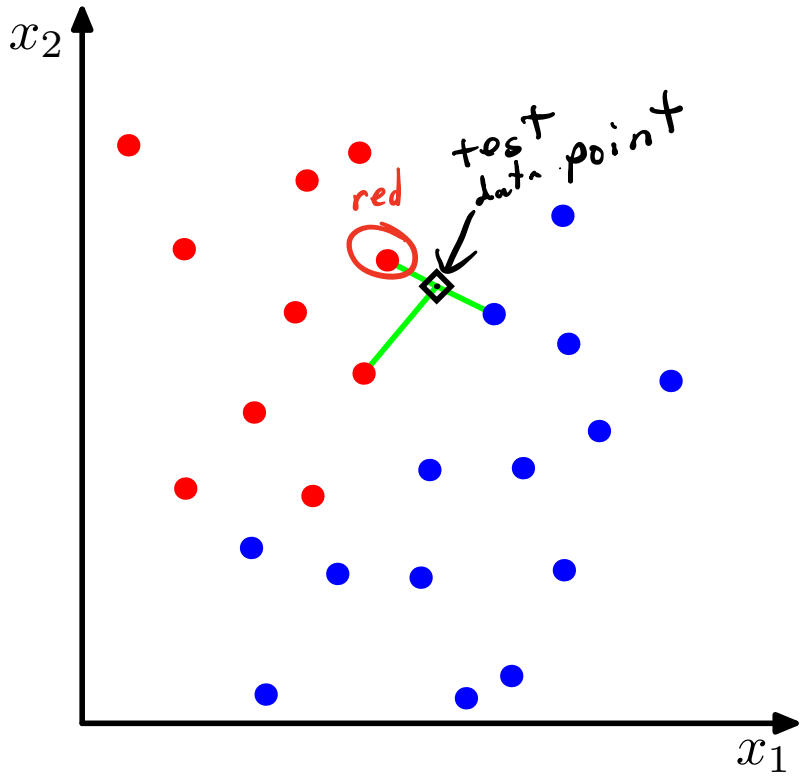
$$y = f(\boldsymbol{x}) = y_{\text{nn}(\boldsymbol{x})}$$

Example: if $\text{nn}(\boldsymbol{x}) = 2$, then $y_{\text{nn}(\boldsymbol{x})} = y_2$, which is the label of the 2nd data point.

2nd training data point

Visual Example

In this 2-dimensional example, the nearest point to $\boldsymbol{x}$ is a red training instance, thus, $\boldsymbol{x}$ will be labeled as red.



(a)

How to Measure "Nearness"?

Previously, we used Euclidean distance

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$$

We can also use alternative distances

- E.g., the ℓ_1 distance (i.e., city block distance, or Manhattan distance):

$$\begin{aligned}\text{nn}(\boldsymbol{x}) &= \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_1 \\ &= \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} |x_d - x_{nd}|\end{aligned}$$

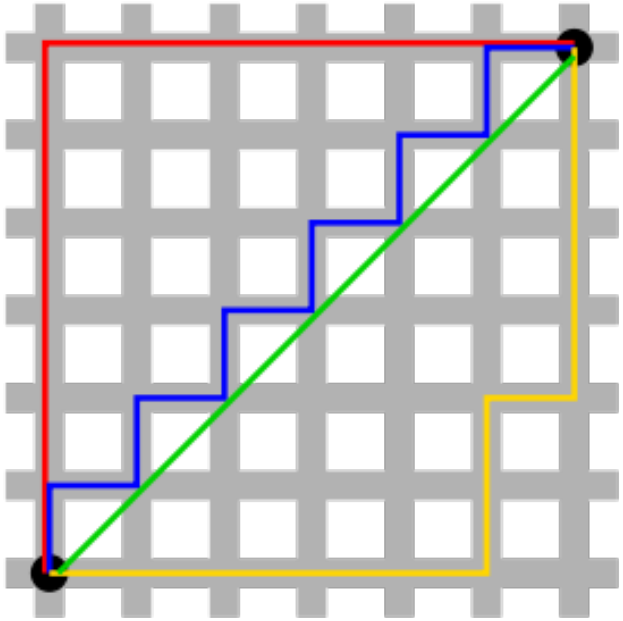


Figure 2: Green line is Euclidean distance. Red, Blue, and Yellow lines are L_1 distance

How to Measure "Nearness"?

Previously, we used Euclidean distance

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$$

We can also use alternative distances

- E.g., the ℓ_1 distance (i.e., city block distance, or Manhattan distance):

$$\begin{aligned}\text{nn}(\boldsymbol{x}) &= \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_1 \\ &= \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} |x_d - x_{nd}|\end{aligned}$$

- Or, the ℓ_∞ (supremum) distance:

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \max_d |x_d - x_{nd}|$$

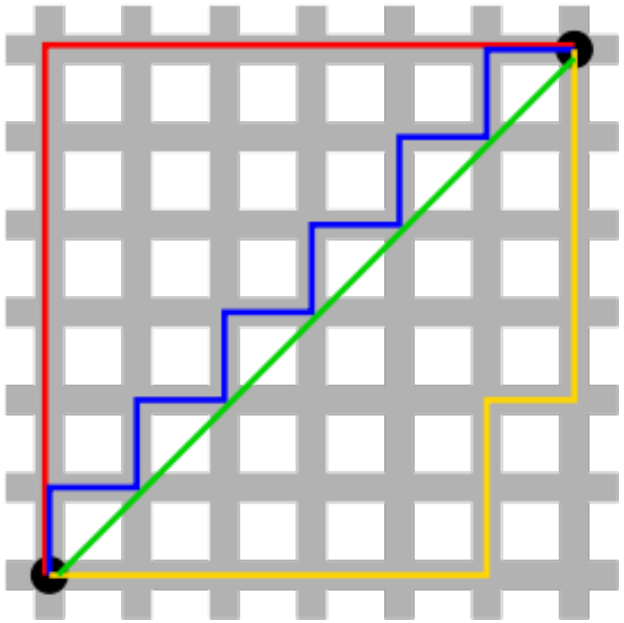


Figure 2: Green line is Euclidean distance. Red, Blue, and Yellow lines are L_1 distance

Nearest Neighbors for Regression

Recall the nearest neighbor of a (training or test) data point:

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\mathrm{nn}(\boldsymbol{x})}$$

where $\mathrm{nn}(\boldsymbol{x}) \in [\mathrm{N}] = \{1, 2, \cdots, \mathrm{N}\}$ indexes a training instance

$$\mathrm{nn}(\boldsymbol{x}) = \mathrm{argmin}_{n \in [\mathrm{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2 = \mathrm{argmin}_{n \in [\mathrm{N}]} \sum_{d=1}^{\mathrm{D}} (x_d - x_{nd})^2$$

Euclidean

Nearest Neighbors for Regression

Recall the nearest neighbor of a (training or test) data point:

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\text{nn}(\boldsymbol{x})}$$

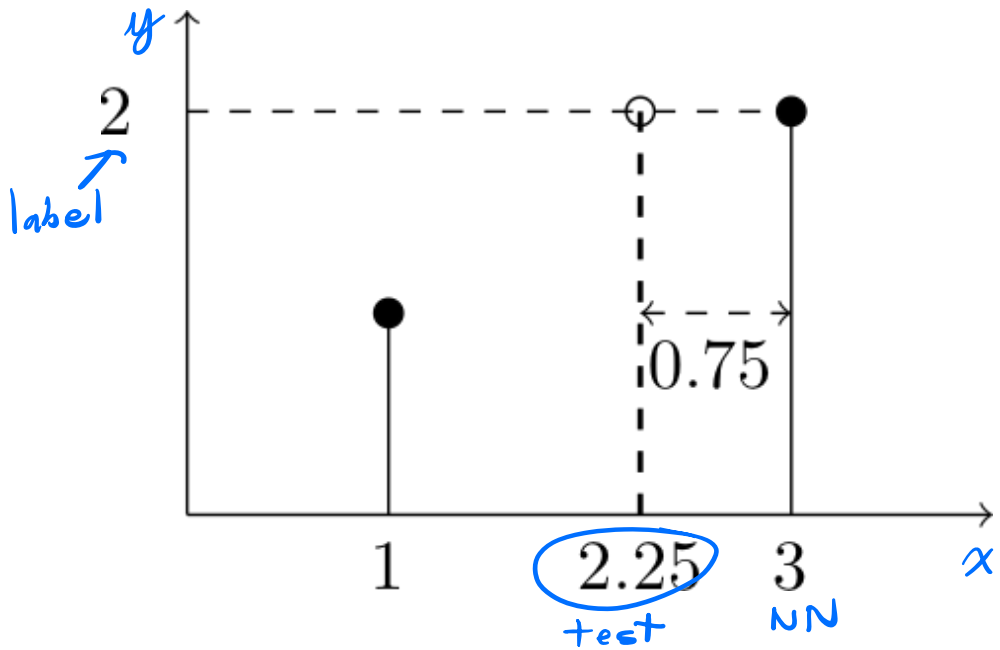
where $\text{nn}(\boldsymbol{x}) \in [\text{N}] = \{1, 2, \cdots, \text{N}\}$ indexes a training instance

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2 = \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} (x_d - x_{nd})^2$$

Regression rule

$$y = f(\boldsymbol{x}) = y_{\text{nn}(\boldsymbol{x})}$$

Label **x** with the label of its nearest neighbor!



Parametric vs. Nonparametric, Revisited

Nonparametric models make predictions directly based on the data $\mathcal{D}$ (without learning any parametric model like the linear decision boundary in SVM/logistic regression).

- Parametric models are often simpler and faster to learn, but can sometimes be a poor fit.
- Nonparametric models are more complex and computationally expensive, but can learn more flexible patterns.

Parametric vs. Nonparametric, Revisited

Nonparametric models make predictions directly based on the data $\mathcal{D}$ (without learning any parametric model like the linear decision boundary in SVM/logistic regression).

- Parametric models are often simpler and faster to learn, but can sometimes be a poor fit.
- Nonparametric models are more complex and computationally expensive, but can learn more flexible patterns.

How does this manifest for nearest neighbors?

- Nearest neighbors often learns a *highly nonlinear* decision boundary.

Parametric vs. Nonparametric, Revisited

Nonparametric models make predictions directly based on the data $\mathcal{D}$ (without learning any parametric model like the linear decision boundary in SVM/logistic regression).

- Parametric models are often simpler and faster to learn, but can sometimes be a poor fit.
- Nonparametric models are more complex and computationally expensive, but can learn more flexible patterns.

How does this manifest for nearest neighbors?

- Nearest neighbors often learns a *highly nonlinear* decision boundary.
- But, we need to compare the test data point to *every sample in the training dataset*, which is *computationally expensive*.

Outline

1. Recap: SVMs
2. Parametric and Nonparametric Models
3. Nearest Neighbor Classification
4. K Nearest Neighbors Classification
5. Practical Aspects of NN
6. Midterm Review

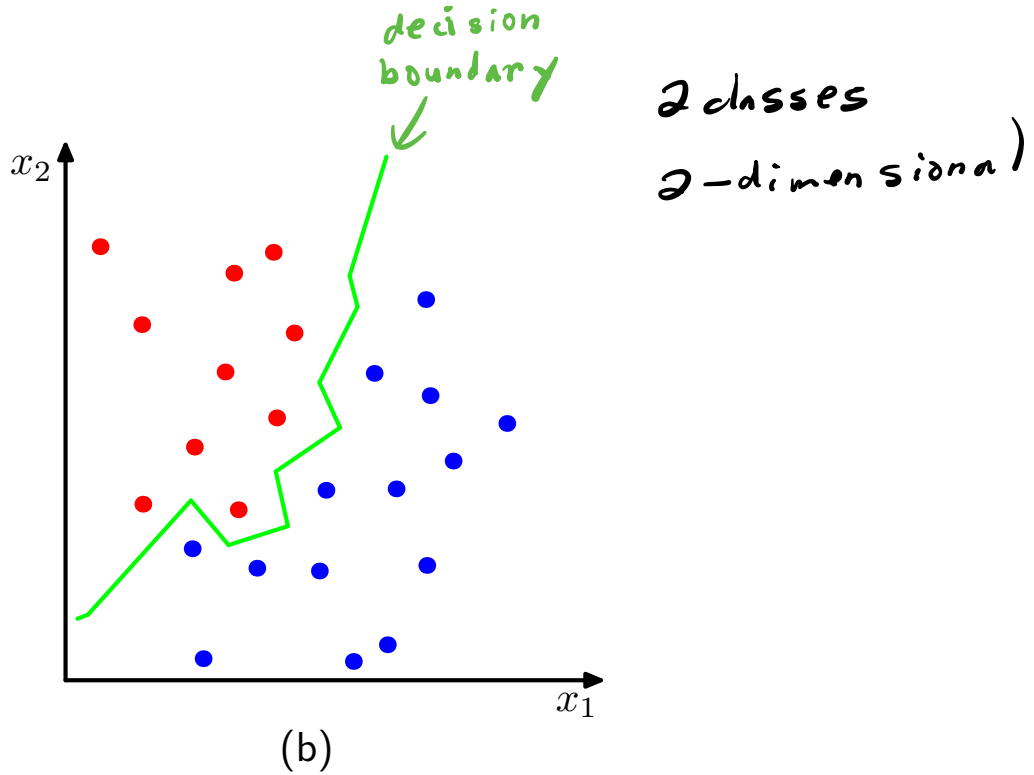
K Nearest Neighbors Classification

Drawbacks of Using One Nearest Neighbor

- What if the nearest neighbor of the test point is an outlier?
- Relying on one nearest neighbor makes the decision boundary susceptible to outliers

Drawbacks of Using One Nearest Neighbor

- What if the nearest neighbor of the test point is an outlier?
- Relying on one nearest neighbor makes the decision boundary susceptible to outliers
- Even without outliers, it results in a complex, jagged decision boundary



(b)

Solution: Use K nearest neighbors and combine their labels

K-Nearest Neighbor (KNN) Classification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $\text{nn}_1(\boldsymbol{x}) = \text{argmin}_{n \in [N]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$

test pt.

training points

K-Nearest Neighbor (KNN) Classification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $\text{nn}_1(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 2nd-nearest neighbor: $\text{nn}_2(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$

2nd closest

K-Nearest Neighbor (KNN) Classification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $\text{nn}_1(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 2nd-nearest neighbor: $\text{nn}_2(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 3rd-nearest neighbor: $\text{nn}_3(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x}) - \text{nn}_2(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$

K-Nearest Neighbor (KNN) Classification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $\text{nn}_1(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 2nd-nearest neighbor: $\text{nn}_2(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 3rd-nearest neighbor: $\text{nn}_3(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x}) - \text{nn}_2(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$

k: positive integer

The set of K-nearest neighbors

$$\text{knn}(\boldsymbol{x}) = \{\text{nn}_1(\boldsymbol{x}), \text{nn}_2(\boldsymbol{x}), \cdots, \text{nn}_K(\boldsymbol{x})\}$$

Let $\boldsymbol{x}(i) = \boldsymbol{x}_{\text{nn}_i(\boldsymbol{x})}$, then ranks closeness of k nearest neighbors

$$\|\boldsymbol{x} - \boldsymbol{x}(1)\|_2^2 \leq \|\boldsymbol{x} - \boldsymbol{x}(2)\|_2^2 \cdots \leq \|\boldsymbol{x} - \boldsymbol{x}(K)\|_2^2$$

How to Classify with K Neighbors?

K nearest neighbors

Classification rule

- Every neighbor votes: suppose y_n (the true label) for $\boldsymbol{x}_n$ is c, then
  - vote for c is 1
  - vote for $c' \neq c$ is 0

How to Classify with K Neighbors?

Classification rule

- Every neighbor votes: suppose y_n (the true label) for $\boldsymbol{x}_n$ is c, then
  - vote for c is 1
  - vote for $c' \neq c$ is 0

  We use the *indicator function* $\mathbb{I}(y_n == c)$ to represent the votes.
- Aggregate everyone's vote

$$v_c = \sum_{n \in \text{knn}(\boldsymbol{x})} \mathbb{I}(y_n == c), \quad \forall \quad c \in [\text{C}]$$

# of K nearest neighbors in class C

How to Classify with K Neighbors?

Classification rule

- Every neighbor votes: suppose y_n (the true label) for $\boldsymbol{x}_n$ is c, then
  - vote for c is 1
  - vote for $c' \neq c$ is 0

  We use the *indicator function* $\mathbb{I}(y_n == c)$ to represent the votes.
- Aggregate everyone's vote

$$v_c = \sum_{n \in \text{knn}(\boldsymbol{x})} \mathbb{I}(y_n == c), \quad \forall \quad c \in [\text{C}]$$

- Label with the majority, breaking ties arbitrarily

$$y = f(\boldsymbol{x}) = \arg\max_{c \in [\text{C}]} v_c$$

if 2 classes, choose k to be odd

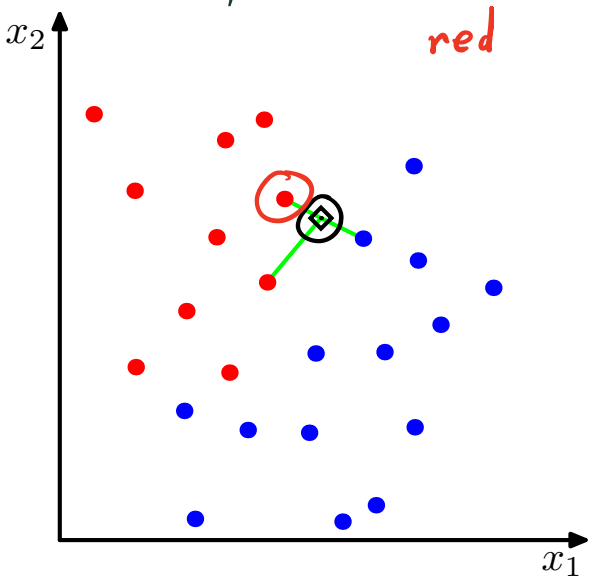
Example

2 dimensional data

2 classes : red / blue

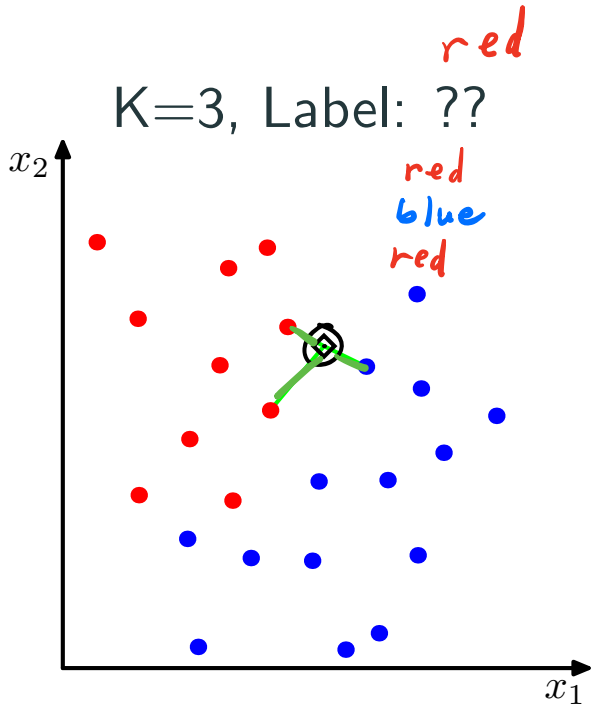
Euclidean distance

K=1, Label: ??



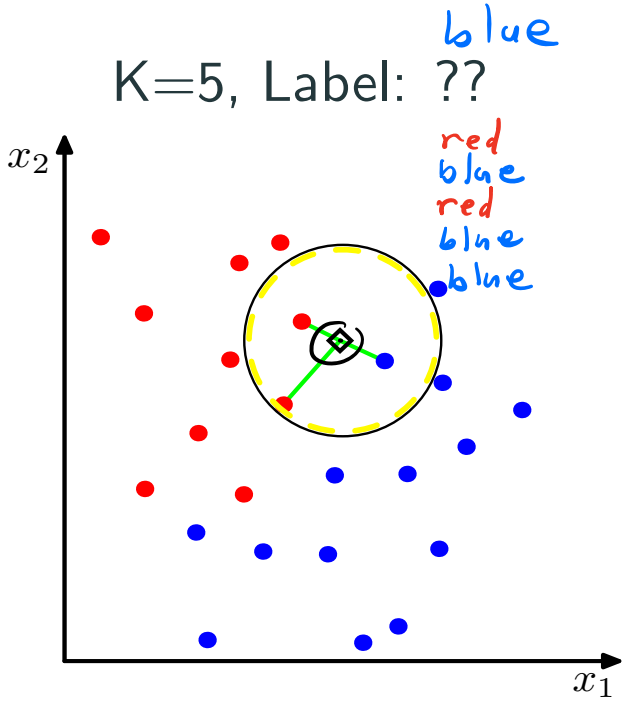
(a)

K=3, Label: ??



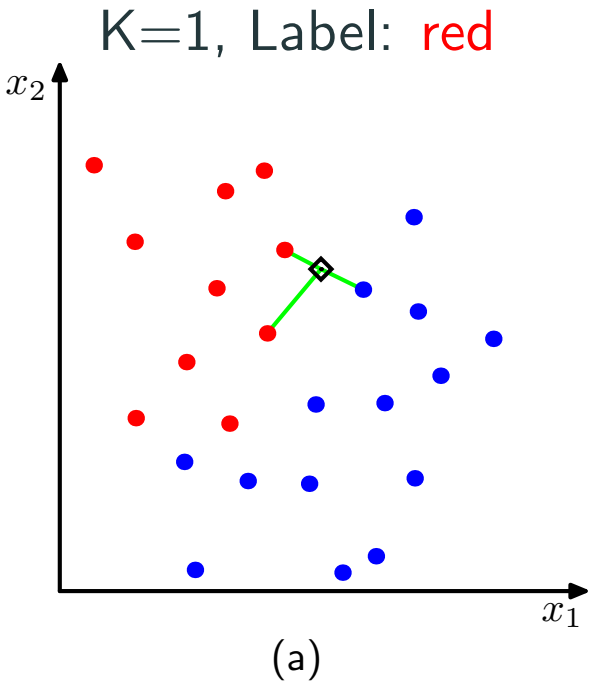
(a)

K=5, Label: ??



(a)

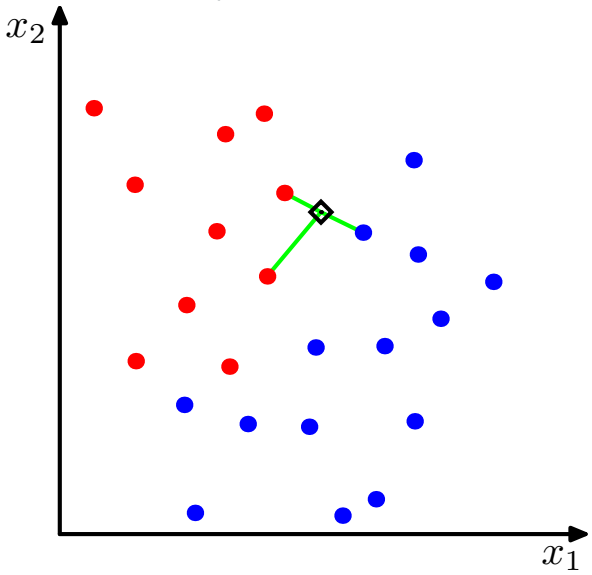
Example



(a)

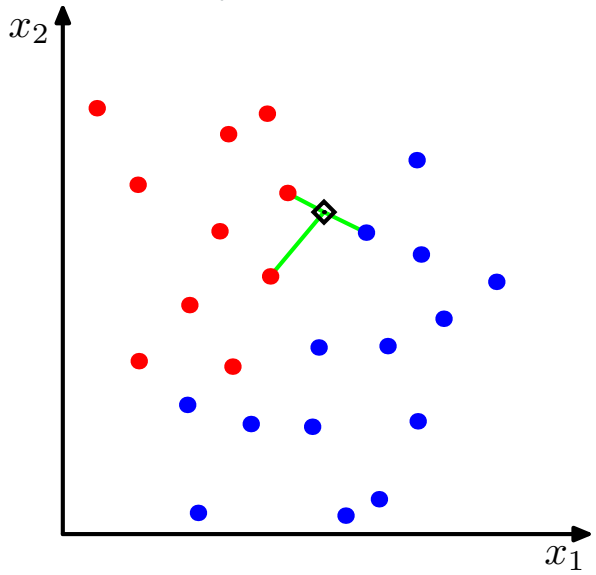
Example

K=1, Label: red



(a)

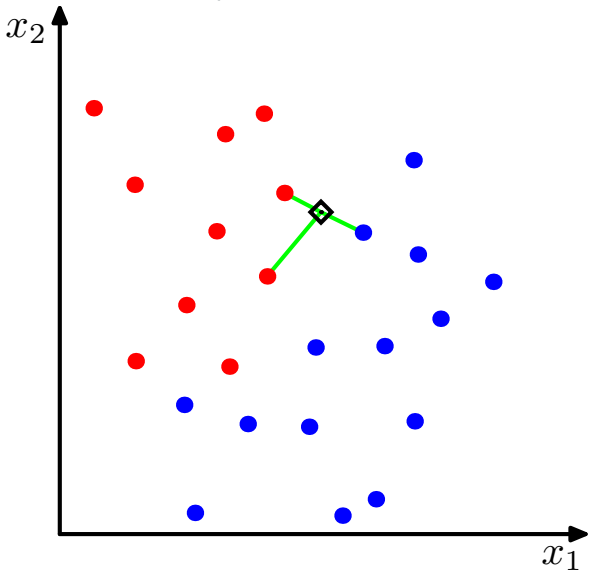
K=3, Label: red



(a)

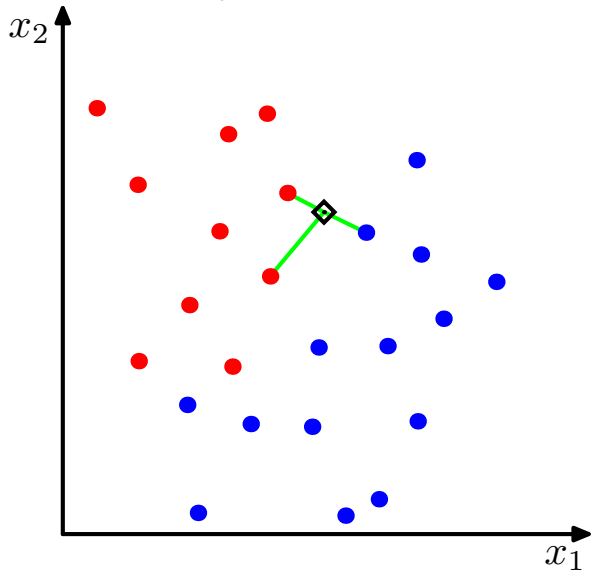
Example

K=1, Label: red



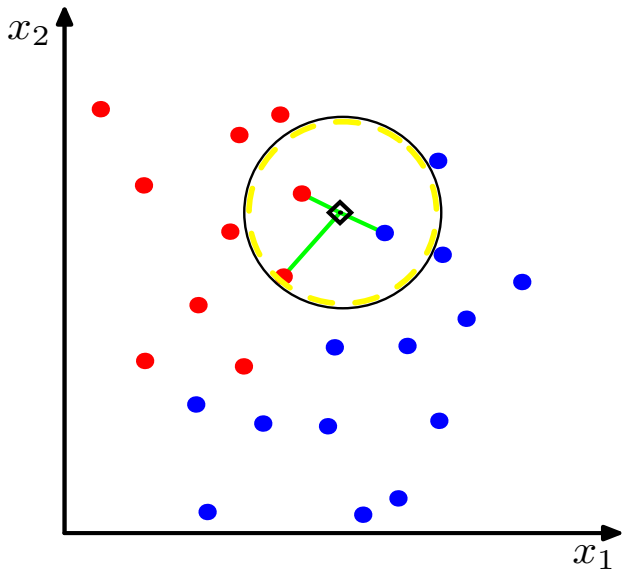
(a)

K=3, Label: red



(a)

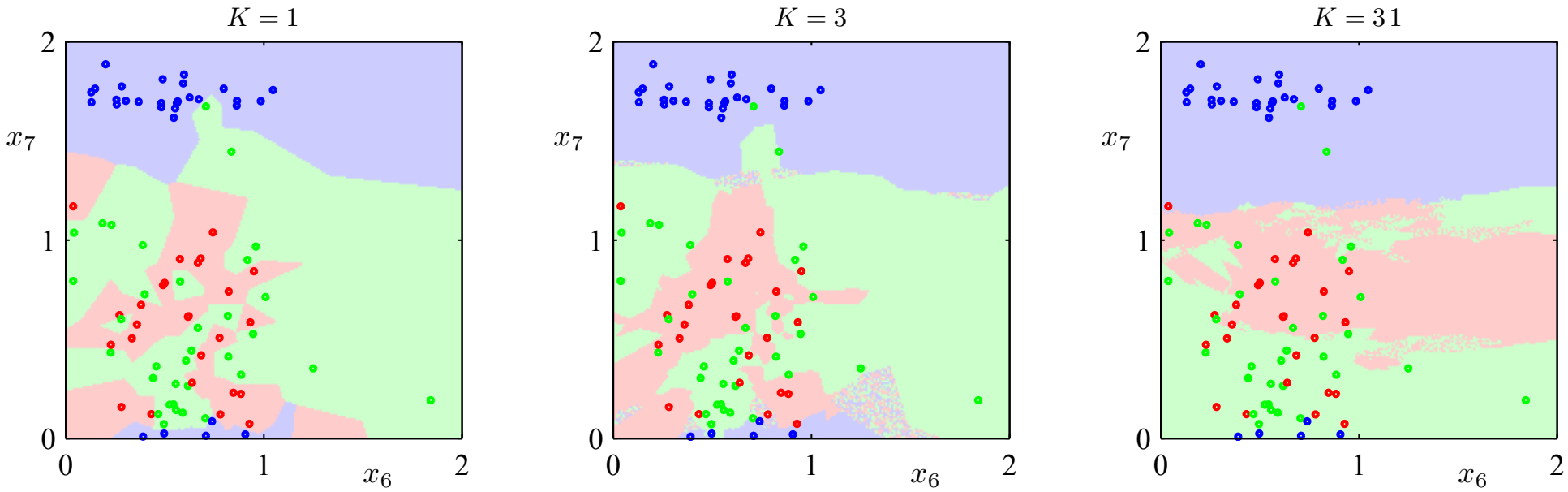
K=5, Label: blue



(a)

How to Choose an Optimal K?

3 classes
2-D data



- When K increases, the decision boundary becomes smoother and less susceptible to outliers.
- However, if K is too large, it can also lead to misclassification as we are taking votes from faraway training points.

K = # training data points

$K \leq \frac{\#\text{training data points}}{\#\text{classes}}$

How to Do Regression with K Neighbors?

- We need a way to aggregate labels from each of the neighbors.
- Average the labels associated with the K points.

$$\hat{y} = \frac{1}{K} \sum_{n \in knn(\mathbf{x})} y_n$$

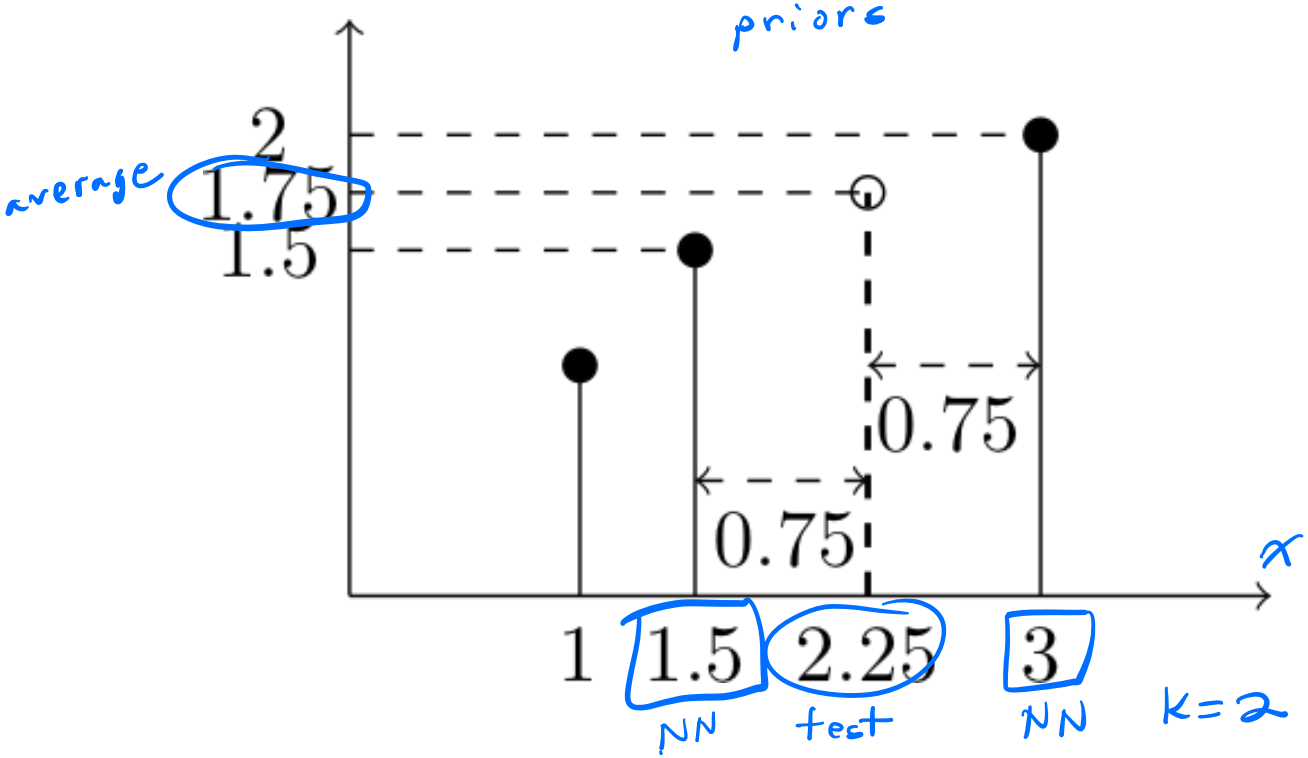
How to Do Regression with K Neighbors?

- We need a way to aggregate labels from each of the neighbors.
- Average the labels associated with the K points.

$$\hat{y} = \frac{1}{K} \sum_{n \in knn(\mathbf{x})} y_n$$

weighted average

prior



Pros and Cons of Nearest Neighbors

Advantages of NNC

- Simple and easy to implement – just compute distances, no optimization required
- Can learn complex decision boundaries

Pros and Cons of Nearest Neighbors

Advantages of NNC

- Simple and easy to implement – just compute distances, no optimization required
- Can learn complex decision boundaries

Disadvantages of NNC

lp distances
D-dim. inputs

- Computationally intensive for large-scale problems: $O(\mathrm{N}D)$ for labeling a data point.
- We need to "carry" the training data around. Without it, we cannot do classification.
- Choosing the right distance measure and K can be difficult.

Pros and Cons of Nearest Neighbors

Advantages of NNC

- Simple and easy to implement – just compute distances, no optimization required
- Can learn complex decision boundaries

Disadvantages of NNC

- Computationally intensive for large-scale problems: $O(\mathrm{N}D)$ for labeling a data point.
- We need to "carry" the training data around. Without it, we cannot do classification.
- Choosing the right distance measure and K can be difficult.
- Relies on the existence of training data points "close" to test points.
- Can break down if the classes are unbalanced.

Practical Aspects of NN

Hyperparameters in NN

Two crucial choices for NN

- Choosing K, i.e., the number of nearest neighbors (default is 1)

Hyperparameters in NN

Two crucial choices for NN

- Choosing K, i.e., the number of nearest neighbors (default is 1)
- Choosing the right distance measure (default is Euclidean distance). Alternatively, can use L_1 distance, or more generally, the following L_p distance measure

$$\|\boldsymbol{x} - \boldsymbol{x}_n\|_p = \left(\sum_d |x_d - x_{nd}|^p \right)^{1/p}$$

difference in feature values

for $p \geq 1$.

- Another popular choice in practice is the cosine distance $\frac{\boldsymbol{x}^T \boldsymbol{x}_n}{\|\boldsymbol{x}\|_2 \|\boldsymbol{x}_n\|_2}$.

HW 2

Hyperparameters in NN

Two crucial choices for NN

- Choosing K, i.e., the number of nearest neighbors (default is 1)
- Choosing the right distance measure (default is Euclidean distance). Alternatively, can use L_1 distance, or more generally, the following L_p distance measure

$$\|\boldsymbol{x} - \boldsymbol{x}_n\|_p = \left(\sum_d |x_d - x_{nd}|^p \right)^{1/p}$$

  for $p \geq 1$.
- Another popular choice in practice is the cosine distance $\frac{\boldsymbol{x}^T \boldsymbol{x}_n}{\|\boldsymbol{x}\|_2 \|\boldsymbol{x}_n\|_2}$.

These are not specified by the algorithm itself — use (cross-)validation!

Preprocessing Data

Normalize data to have zero mean and unit standard deviation in each dimension

- Compute the means and standard deviations in each feature

$$\bar{x}_d = \frac{1}{N} \sum_n x_{nd}, \qquad s_d^2 = \frac{1}{N-1} \sum_n (x_{nd} - \bar{x}_d)^2$$

- Scale the feature accordingly

$$x_{nd} \leftarrow \frac{x_{nd} - \bar{x}_d}{s_d}$$

Preprocessing Data

Normalize data to have zero mean and unit standard deviation in each dimension

- Compute the means and standard deviations in each feature

$$\bar{x}_d = \frac{1}{\mathrm{N}} \sum_n x_{nd}, \qquad s_d^2 = \frac{1}{N-1} \sum_n (x_{nd} - \bar{x}_d)^2$$

- Scale the feature accordingly

$$x_{nd} \leftarrow \frac{x_{nd} - \bar{x}_d}{s_d}$$

Many other ways of normalizing data — you would need/want to try different ones and pick among them using (cross) validation

You Should Know

- The differences between parametric and non-parametric learning models
- How to implement K-nearest neighbors for regression and classification
- Practical aspects of NNC, such as tuning hyperparameters (K, distance metric) and feature scaling.

normalization

Midterm Review

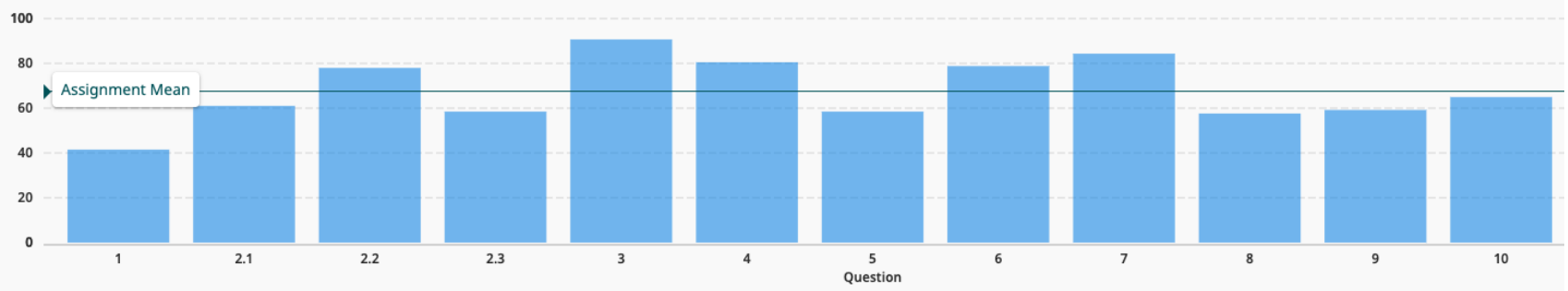
Concepts You Should Know

This is a quick overview of important concepts/methods/models.

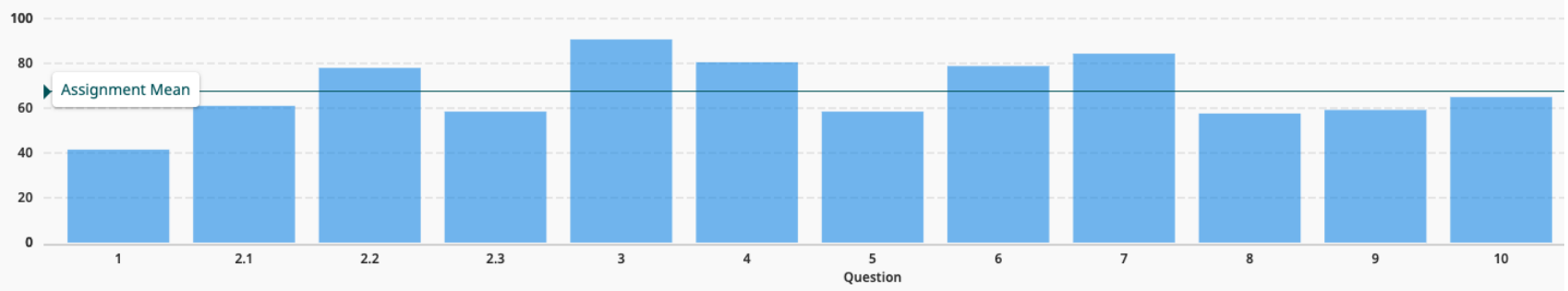
through today's lecture

- MLE/MAP: how to find the likelihood of one or more observations, how to use a prior distribution, how to optimize the (log-)likelihood
- Linear regression: formulation, how it relates to MLE/MAP, feature scaling, ridge regression, gradient descent, nonlinear basis functions
- Bias-variance trade-off: bias and variance, overfitting, cross-validation
- Naïve Bayes: naïve classification rule, why it is naïve, Laplacian smoothing
- Logistic regression: generative vs. discriminative models, formulation, how it relates to MLE, comparison to naïve Bayes, sigmoid function, cross-entropy function, nonlinear boundaries
- Multi-class: one-vs-all and one-vs-one approaches, softmax function/multinomial logistic regression
- SVMs: hinge loss formulation, hard and soft margin formulations, support vectors, dual of the SVM problem, kernel functions
- Nearest neighbors: parametric vs. nonparametric models, k-nearest neighbors, tuning hyperparameters, feature scaling

Mini-Exam 1


100
80
60
40
20
0
Assignment Mean
1
2.1
2.2
2.3
3
4
5
6
7
8
9
10
Question

Mini-Exam 1



Problem 1: *[1 points]* A data scientist trained a regression model for house price prediction. After training the model, the ground-truth price labels were lost due to a storage failure. The data scientist cannot estimate the model's variance without these labels.

- ◯ True
- ◯ False

Solution: True. Model variance cannot be estimated without ground-truth labels because we need the labels to retrain on different resampled versions of the training data.

Mini-Exam 1

Problem 5: *[1 points]* You are given a design matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$. First, you apply the nonlinear basis function $\phi(x) = x^2$ elementwise to $\mathbf{X}$ to obtain $\mathbf{\Phi}$. Then you compute the ridge regression solution

$$\mathbf{w}_{\text{ridge}} = (\mathbf{\Phi}^\top \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^\top \mathbf{y},$$

where $\lambda > 0$ is a given regularization parameter and $\mathbf{y}$ denotes the target vector. Recall that the ordinary least squares (OLS) solution $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ has computational complexity $O(ND^2 + D^3)$. OLS
Even though ridge regression requires the additional operation of adding $\lambda \mathbf{I}$ to $\mathbf{\Phi}^\top \mathbf{\Phi}$ before inversion, and we apply a nonlinear transformation $\phi(x) = x^2$ to every element of $\mathbf{X}$ to obtain $\mathbf{\Phi}$, the dominant computational complexity of computing $\mathbf{w}_{\text{ridge}}$ remains $O(ND^2 + D^3)$.

- ◯ True
- ◯ False

D×D

ridge regression

Solution: True.
Although ridge regression requires adding $\lambda \mathbf{I}$ (costing $O(D^2)$) and applying ϕ requires $O(ND)$ operations, both are dominated by the existing $O(ND^2 + D^3)$ complexity. The matrix multiplication $\mathbf{\Phi}^\top \mathbf{\Phi}$ still costs $O(ND^2)$, and the inversion still costs $O(D^3)$. Therefore, the dominant complexity remains $O(ND^2 + D^3)$.

$O(ND^2 + D^3 + D^2 + ND)$

new operations

Mini-Exam 1

Problem 8: *[2 points]* For Stochastic Gradient Descent (SGD) and full-batch gradient descent (GD), which of the following statements are true? **More than one option can be true.**

- ☐ In each iteration of SGD, one sample is drawn uniformly at random without replacement to ensure that the gradient is an unbiased estimate of the gradient used in full-batch GD.
- ☐ In each iteration of SGD, one sample is drawn uniformly at random with replacement to ensure that the gradient is an unbiased estimate of the gradient used in full-batch GD. iterations are ind.
- ☐ SGD can get stuck in local minima, whereas GD is guaranteed to reach the global minimum of a non-convex objective function.
- ☐ GD can get stuck in local minima, whereas SGD is guaranteed to reach the global minimum of a non-convex objective function.

Solution: B

- A is False; because without replacement sampling will lead to biased stochastic gradients
- B is True; because with replacement sampling will lead to unbiased stochastic gradients
- C and D are both False; SGD and GD can both get stuck in local minima

Mini-Exam 1

Problem 9

$X \in \{1, 2, 3\}$

(a) Write down $\Pr(X = 1)$, $\Pr(X = 2)$, and $\Pr(X = 3)$ in terms of p and δ.

Solution: Since users stop at first success,

$$\Pr(X = 1) = p, \qquad \Pr(X = 2) = (1 - p)(p + \delta), \qquad \Pr(X = 3) = (1 - p)(1 - (p + \delta))$$

n_1 n_2 n_3

(b) Write down the log-likelihood function $\ell(p)$, and derive the first-order condition that the MLE must satisfy, namely $\ell'(p) = 0$ (no need to solve it).

likelihood: $p^{n_1} \cdot [(1-p)(p+\delta)]^{n_2} \cdot [(1-p)(1-p-\delta)]^{n_3}$

Solution: The log-likelihood is

$$\ell(p) = n_1 \log p + n_2\big(\log(1 - p) + \log(p + \delta)\big) + n_3\big(\log(1 - p) + \log(1 - p - \delta)\big)$$

Differentiating and setting $\ell'(p) = 0$ gives the first-order condition

$$\ell'(p) = \frac{n_1}{p} - \frac{n_2 + n_3}{1 - p} + \frac{n_2}{p + \delta} - \frac{n_3}{1 - p - \delta} = 0$$

Gradescope Quiz 5

Suppose that you tried to train a logistic regression model on a dataset $\mathcal{D}$, but you found that the optimal model classified some of your training data points incorrectly. You tried to fix this by using nonlinear basis functions to transform the original input features x_1, x_2 into the new features x_1, x_2, x_1^2, x_2^2, but your model trained on the new features still classified some of the training data points incorrectly. high training error

Which of the following changes might ensure that your learned model classifies all of the training data points correctly?

(a) ( ) Train the logistic regression model using the features $x_1, x_2, x_1 + x_2^2$. ✗ x_1, x_2, x_1^2, x_2^2 doesn't work

(b) ( ) Introduce a regularization term into the objective function. ✗ doesn't reduce training error

(c) ( ) Collect new training data points and add them to $\mathcal{D}$. good for test error

(d) (X) None of the above.

Gradescope Quiz 4

Consider a binary classification problem. Suppose that you train a naive Bayes model to solve this problem, using Laplacian smoothing with parameter α to compensate for features that are missing in one class. Recall that to do so, we pretend to have seen each feature α additional times in each class.

As you increase the Laplacian smoothing parameter α, which of the following will happen to your model?

(a) ( ) The model's predictions on a test dataset will become more similar to those made without Laplacian smoothing.

(b) ( ) The model will become equivalent to MAP estimation with a uniform prior distribution.

(c) ( ) The prior probabilities for each class will eventually approach 0.5.

(d) (X) None of the above.